

Kolmogorov-Arnold networks

Ziming Liu, Yixuan Wang, Sachin Vaidya, et al.
talked by Rubtsov Denis

Department of intelligent systems, MIPT
Bayesian Multimodelling course

2025

The Limitations of MLPs

- ▶ Multi-Layer Perceptrons (MLPs) are foundational but have significant drawbacks:
 - ▶ Fixed activation functions on **nodes** (neurons)
 - ▶ Linear transformations W separate from nonlinearities σ
 - ▶ Consume majority of parameters in transformers
 - ▶ Less interpretable without post-analysis tools
- ▶ MLP formulation:

$$\text{MLP}(x) = (W_{L-1} \circ \sigma \circ W_{L-2} \circ \sigma \circ \cdots \circ W_1 \circ \sigma \circ W_0)x$$

Kolmogorov-Arnold Representation Theorem

Theorem (Kolmogorov-Arnold, 1956-1957)

If $f : [0, 1]^n \rightarrow \mathbb{R}$ is a multivariate continuous function, then f can be written as a finite composition of continuous functions of a single variable and the binary operation of addition:

$$f(x) = f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

where $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$.

- ▶ Kolmogorov and Arnold showed that the only true multivariate function is the sum, since every other function can be written using univariate functions and summing.
- ▶ Previously considered impractical due to non-smooth inner functions

Intuitive picture of KART

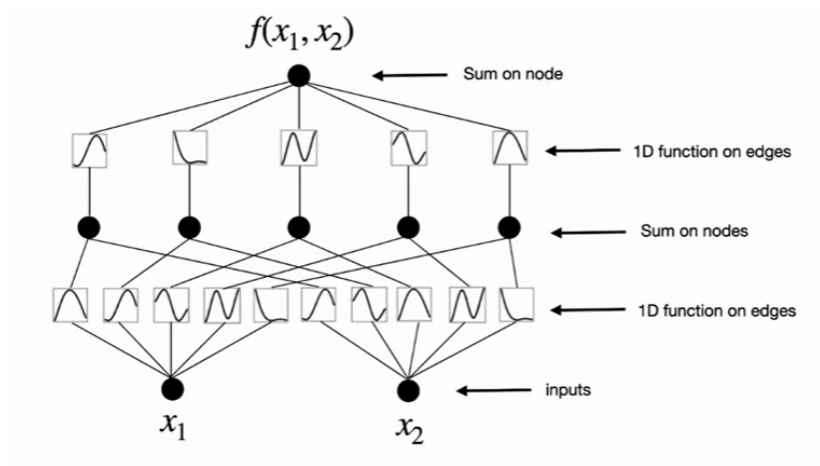


Рис.: MLP vs. KAN architecture

Intuitive picture of KART

image (15).png

KANs: A New Paradigm

- ▶ **Kolmogorov-Arnold Networks (KANs)** leverage the theorem with modern deep learning
- ▶ Key differences from MLPs:
 - ▶ Learnable activation functions on **edges** (weights)
 - ▶ No linear weight matrices
 - ▶ Each "weight" is a univariate function parametrized as a spline
 - ▶ Nodes perform simple summation
- ▶ Combines strengths of splines (accuracy in 1D) and MLPs (compositional structure learning)

MLP vs KAN

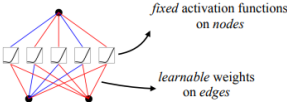
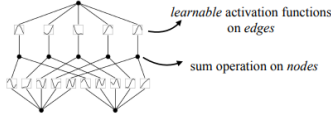
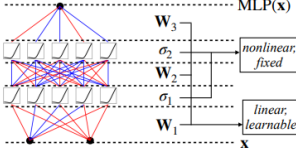
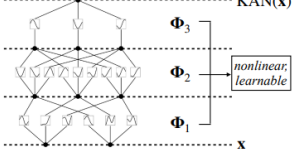
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a) 	(b) 
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c) 	(d) 

Рис.: MLP vs. KAN architecture

Mathematical Formulation

- ▶ KAN layer with n_{in} inputs and n_{out} outputs:

$$\Phi = \{\phi_{q,p}\}, \quad p = 1, \dots, n_{\text{in}}, \quad q = 1, \dots, n_{\text{out}}$$

- ▶ Forward pass for layer l :

$$x_{l+1,j} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i}), \quad j = 1, \dots, n_{l+1}$$

- ▶ Matrix form:

$$\mathbf{x}_{l+1} = \begin{pmatrix} \phi_{l,1,1}(\cdot) & \cdots & \phi_{l,1,n_l}(\cdot) \\ \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1}(\cdot) & \cdots & \phi_{l,n_{l+1},n_l}(\cdot) \end{pmatrix} \mathbf{x}_l$$

Activation Function Parametrization

- ▶ Each activation function $\phi(x)$ is parametrized as:

$$\phi(x) = w_b b(x) + w_s \cdot \text{spline}(x)$$

- ▶ Base function: $b(x) = \text{silu}(x) = x/(1 + e^{-x})$
- ▶ Spline component: $\text{spline}(x) = \sum_i c_i B_i(x)$
 - ▶ $B_i(x)$: B-spline basis functions
 - ▶ c_i : Trainable coefficients
- ▶ Grid extension: Refine spline grids without retraining from scratch

Splines

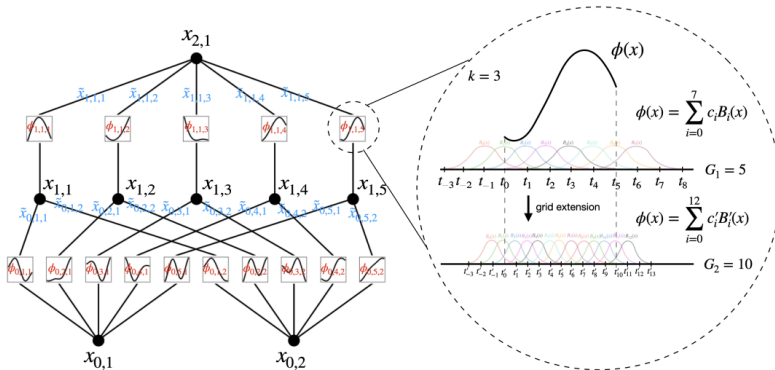


Рис.: B-spline parametrization of activation functions

Approximation Theory for KANs

Theorem (Approximation Theory, KAT)

Let $\mathbf{x} = (x_1, \dots, x_n)$. Suppose $f(\mathbf{x})$ admits representation

$$f = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_1 \circ \Phi_0)\mathbf{x}$$

with $(k+1)$ -times continuously differentiable $\Phi_{l,i,j}$. Then there exists constant C such that:

$$\|f - (\Phi_{L-1}^G \circ \dots \circ \Phi_0^G)\mathbf{x}\|_{C^m} \leq CG^{-k-1+m}$$

for any $0 \leq m \leq k$.

- ▶ **Breaks curse of dimensionality:** Error rate independent of dimension
- ▶ **Scaling exponent:** $\alpha = k + 1$ (e.g., $\alpha = 4$ for cubic splines)

Neural Scaling Laws

- ▶ Test loss ℓ scales with parameters N as $\ell \propto N^{-\alpha}$
- ▶ Comparison of scaling exponents:
 - ▶ **Sharma & Kaplan:** $\alpha = (k + 1)/d$ (suffers from COD)
 - ▶ **Michaud et al.:** $\alpha = (k + 1)/2$ (max arity 2)
 - ▶ **Poggio et al.:** $\alpha = m/2$ (compositional sparsity)
 - ▶ **KANs:** $\alpha = k + 1$ (e.g., $\alpha = 4$ for cubic splines)
- ▶ KANs achieve the **best possible scaling exponent**

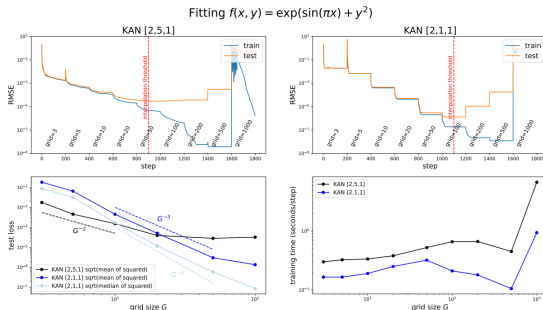


Figure 2.3: We can make KANs more accurate by grid extension (fine-graining spline grids). Top left (right): training dynamics of a $[2, 5, 1]$ ($[2, 1, 1]$) KAN. Both models display staircases in their loss curves, i.e., loss suddenly drops then plateaus after grid extension. Bottom left: test RMSE follows scaling laws against grid size G . Bottom right: training time scales favorably with grid size G .

PDE Solving: Poisson Equation

- ▶ Consider Poisson equation on $\Omega = [-1, 1]^2$:

$$u_{xx} + u_{yy} = f \quad \text{in } \Omega, \quad u = 0 \quad \text{on } \partial\Omega$$

- ▶ Physics-Informed Neural Networks (PINNs) framework:

$$\text{loss}_{\text{pde}} = \alpha \text{loss}_i + \text{loss}_b$$

where

$$\text{loss}_i = \frac{1}{n_i} \sum_{i=1}^{n_i} |u_{xx}(z_i) + u_{yy}(z_i) - f(z_i)|^2$$

$$\text{loss}_b = \frac{1}{n_b} \sum_{i=1}^{n_b} u^2$$

PDE Results: KANs vs MLPs

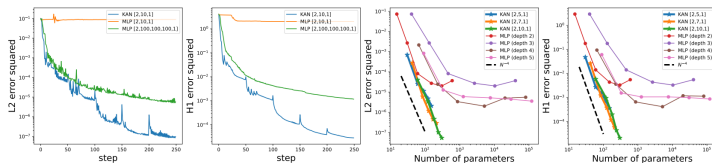
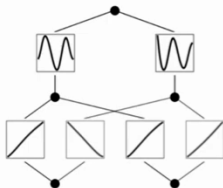


Figure 3.3: The PDE example. We plot L2 squared and H1 squared losses between the predicted solution and ground truth solution. First and second: training dynamics of losses. Third and fourth: scaling laws of losses against the number of parameters. KANs converge faster, achieve lower losses, and have steeper scaling laws than MLPs.

- ▶ **2-Layer width-10 KAN:**
 - ▶ 100× more accurate than 4-Layer width-100 MLP
 - ▶ 100× more parameter efficient (10^2 vs 10^4 parameters)
- ▶ KANs achieve lower errors with fewer parameters
- ▶ Steeper scaling laws in both L^2 and H^1 norms

PDE Results



After setting all to be symbolic, we further train the model (affine parameters are still trainable). The model can now reach machine precision!

```
train()
```

```
pde loss: 1.37e-16 | bc loss: 3.89e-18 | l2: 7.38e-18 : 100%| 20/20 [00:07<00:
```

Print out the symbolic formula

```
formula, var = model.symbolic_formula(floating_digit=5)
formula[0]
```

```
0.5 sin (3.14159x1 - 3.14159x2 + 7.85398) - 0.5 sin (3.14159x1 + 3.14159x2 + 1.5708)
```

Simplification Techniques

- **Sparsification:** L1 + entropy regularization

$$\ell_{\text{total}} = \ell_{\text{pred}} + \lambda \left(\mu_1 \sum_{l=0}^{L-1} |\Phi_l|_1 + \mu_2 \sum_{l=0}^{L-1} S(\Phi_l) \right)$$

where $|\phi|_1 = \frac{1}{N_p} \sum_{s=1}^{N_p} |\phi(x^{(s)})|$

- **Pruning:** Remove nodes with low incoming/outgoing scores

$$I_{l,i} = \max_k (|\phi_{l-1,i,k}|_1), \quad O_{l,i} = \max_j (|\phi_{l+1,j,i}|_1)$$

- **Symbolification:** Fix activation functions to symbolic forms

$$\text{fix_symbolic}(l, i, j, f)$$

Supervised Toy Examples

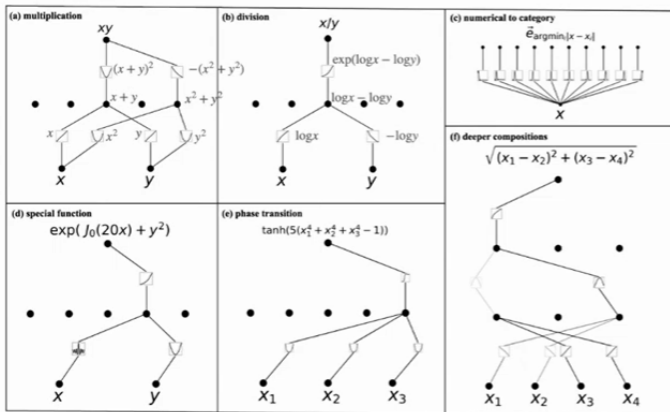
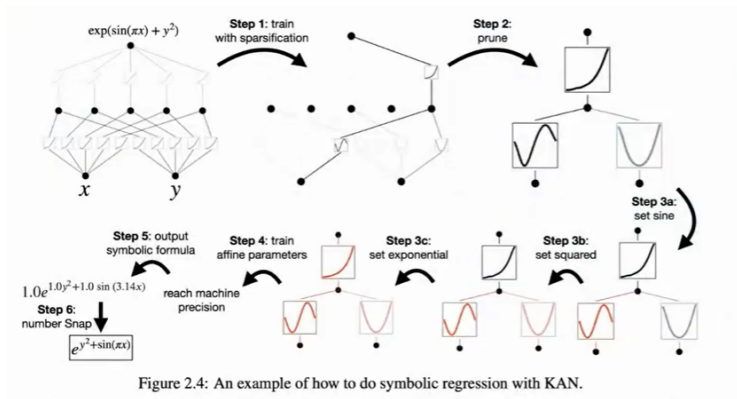


Figure 4.1: KANs are interpretable for simple symbolic tasks

- ▶ (a) Multiplication: $f(x, y) = xy \rightarrow$ learns $2xy = (x + y)^2 - (x^2 + y^2)$
- ▶ (b) Division: $f(x, y) = x/y \rightarrow$ learns $x/y = \exp(\log x - \log y)$

Else one toy example



- ▶ **Generalized Kolmogorov-Arnold Theorem:**
 - ▶ Deeper representations may enable smoother activations
 - ▶ "Kolmogorov-Arnold depth" as function class characterization
- ▶ **Algorithmic Improvements:**
 - ▶ Alternative parametrizations (radial basis functions)
 - ▶ Multi-head activation grouping
 - ▶ Multi-level training strategies
- ▶ **Comparison with UAT:**
 - ▶ UAT: No error bounds, suffers from COD
 - ▶ KAT: Quantifies error, breaks COD for compositional functions

Future Directions & Applications

- ▶ **Mathematical Foundation:**
 - ▶ Deeper Kolmogorov-Arnold representations
 - ▶ Connection to function space theory
- ▶ **Scientific Applications:**
 - ▶ Navier-Stokes equations, density functional theory
 - ▶ "Kansformers": Replace MLPs with KANs in transformers
- ▶ **AI + Science Paradigm:**
 - ▶ KANs as "language" for scientist-AI collaboration
 - ▶ Interpretable function-based communication

When to Use KANs vs MLPs

- ▶ **Use KANs:** Interpretability and accuracy critical, training speed secondary
- ▶ **Use MLPs:** Fast training required, interpretability less important

Should YOU use KANs?

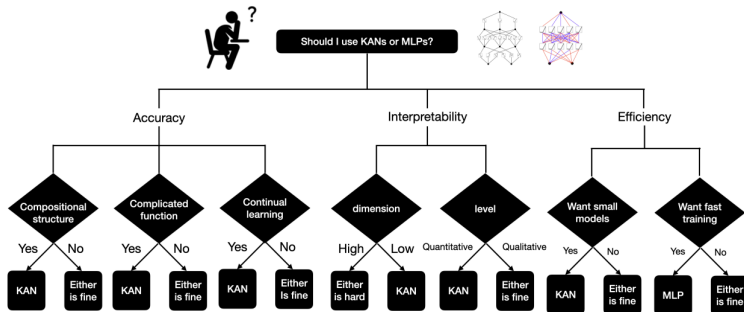


Figure 6.1: Should I use KANs or MLPs?

Thank You

Questions?